

Experiment 'p_4_2_a_min_max_array' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_array

Natural language statement: Write a method that takes a sequence of integers and returns the smallest and largest of the inputs.

Method signature: p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

Ensures

- forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
- forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]
- exists i :: 0 <= i < inputs.Length && min == inputs[i]
- exists i :: 0 <= i < inputs.Length && max == inputs[i]

Requires

- 0 < inputs.Length

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the smallest and largest of
→ the inputs.

The signature should be:

p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < inputs.Length, ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[
→ i], ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i], ensures
→ exists i :: 0 <= i < inputs.Length && min == inputs[i], ensures exists i :: 0 <= i
→ < inputs.Length && max == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme5ULJ9LZgJbBLvibBGpoXu7keZu

Model: gpt-5.5-2026-04-23

Created at: 1780486212

Finish reason: stop

Usage: 412 tokens in, and 588 tokens out

```
{
  min := inputs[0];
  max := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    invariant exists j :: 0 <= j < i && min == inputs[j]
    invariant exists j :: 0 <= j < i && max == inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
    }
    if inputs[i] > max {
      max := inputs[i];
    }
  }
}
```

```

    }
    i := i + 1;
  }
}

```

Final Program

```

method p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)
  requires 0 < inputs.Length
  ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
  ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]
  ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]
  ensures exists i :: 0 <= i < inputs.Length && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    invariant exists j :: 0 <= j < i && min == inputs[j]
    invariant exists j :: 0 <= j < i && max == inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
    }
    if inputs[i] > max {
      max := inputs[i];
    }
    i := i + 1;
  }
}

```

Total Token Usage

Input tokens: 412

Output tokens: 588

Reasoning tokens: 397

Sum of 'total tokens': 1000

Experiment Timings

Overall Experiment started at 1780486211817, ended at 1780486227458, lasting 15641ms (15.64 seconds)

Iteration #1 started at 1780486211820, ended at 1780486227456, lasting 15636ms (15.64 seconds)